Maximizing Energy Efficiency in Spiking Neural Networks: A Dynamic Joint Pruning Framework

Shuo Chen*†‡, Zeshi Liu*, Haihang You*‡
*Institute of Computing Technology, Chinese Academy of Sciences, Bejing, China
†School of Computer Science and Technology, University of Chinese Academy of Sciences, Bejing, China
‡Zhongguancun Laboratory, Bejing, China
{chenshuo20s, liuzeshi, youhaihang}@ict.ac.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132570

***Abstract*—Spiking Neural Networks (SNNs) face increasing challenges for efficient deployment as architectures grow in complexity, necessitating network pruning to improve energy and computational efficiency. Existing pruning methods primarily focus on a single form of sparsity, overlooking the importance of joint pruning, which is critical for minimizing synaptic operations (SOPs) and enhancing energy efficiency. This paper presents a novel dynamic joint pruning framework that leverages both spatiotemporal spike sparsity and weight sparsity to minimize SOPs in SNN inference. Based on a comprehensive analysis of the SOPs model, we introduce an integrated solution that combines a multi-stage masking mechanism for fine-grained neuron firing threshold control, a temporal attention batch normalization (TABN) module with learnable time scaling factors, and a dynamic sparse strategy that adjusts importance coefficients based on real-time computational impact. Experimental results on CIFAR-10, CIFAR-100, and ImageNet validate the effectiveness of proposed framework. Our method achieves up to 126.38× compression ratio of SOPs on CIFAR-10 with minimal accuracy loss, establishing a new state-of-the-art in energy-efficient SNN pruning.**

***Index Terms*—spiking neural network, network pruning, low power computing**

I. Introduction

Spiking Neural Networks (SNNs), regarded as the third generation of neural networks [1], have attracted significant attention for their potential in energy-efficient machine intelligence [2], [3]. By leveraging sparse and event-driven spike signals, SNNs enable low-power and multiplication-free inference on neuromorphic hardware compared to Artificial Neural Networks (ANNs) [4]–[6]. However, to tackle increasingly complex tasks, modern SNN architectures have grown in depth and scale [7], [8], which requires greater computational resources and compromises their low power advantages.

Network pruning has emerged as a promising approach to eliminate redundancy and reduce model size in SNNs. Common techniques mainly focus on pruning synaptic connections, including magnitude-based pruning [9], [10], optimization-driven methods [11], [12], and bio-inspired algorithms [13], [14]. Although these methods achieve high weight sparsity and reduce computational resource requirements, they are less effective at directly reducing the number of synaptic operations (SOPs). SOPs reflect both the computational cost and energy consumption of SNN inference on neuromorphic hardware. Therefore, many works [4]–[6], [11] use SOP-related metrics (e.g., energy consumption per SOP) as indicators for assessing hardware energy efficiency. This indicates that minimizing SOPs is crucial for achieving low-power SNN computing.

To minimize SOPs in SNN inference, we develop a fine-grained SOPs model for SNNs and explore comprehensive pruning strategies. According to the SOPs model, weight sparsity and spatiotemporal spike sparsity are two fundamental factors determining SOP reduction. Additionally, the evolution of importance coefficients during pruning serves as another critical factor in the sparse training process. Previous works have achieved spike number reduction by different approaches [15]–[18]. In particular, sparse regularization as a pruning method directly eliminates model redundancy and achieves high spike sparsity [19]–[21].

However, studies that combine spike and weight sparsity to maximize SOP reduction remain limited. UPSNN [22] and ADMM [23] have preliminarily explored joint pruning with sparse regularization. The former introduces neuron sparsity but not the finer-grained spike sparsity, which directly determines SOP reduction. The latter simply performs weight pruning and activity regularization simultaneously without optimization, making it ineffective in achieving high sparsity during joint pruning. Consequently, neither method achieves the minimization of SOPs. Moreover, even after addressing these issues, joint pruning with sparse regularization still faces challenges: (1) spikes and weights being interdependent during pruning, as spikes are model outputs influenced by weights; (2) the need to balance the sparse strengths of different regularization terms during sparse training.

In this paper, we propose a dynamic joint pruning framework[1] that fully exploits available redundancy to minimize SOPs in SNN inference. Novel techniques are introduced to address the challenges in joint pruning. First, we use a multi-stage masking mechanism to decouple spike sparsity from weights. These masks partition neuron membrane potentials into multiple stages, enabling neurons to adopt different firing thresholds during inference and achieve spatial spike sparsity. Second, to address temporal redundancy, we introduce temporal attention batch normalization (TABN). TABN incorporates a trainable time scaling factor into standard batch normalization, thereby pruning redundant spike activity across time steps. Finally, we employ a dynamic regularization strategy for joint sparse training. Coefficients that reflect the importance of pruning candidates are dynamically updated based on the SOPs model to adjust sparse strengths. Experimental results demonstrate that the proposed framework significantly reduces SOPs, lowers energy consumption, and maintains high model accuracy.

In summary, our key contributions are as follows:

- We develop an SOPs model that incorporates the temporal dimension. This model identifies spatiotemporal spike sparsity, weight sparsity, and the evolution of importance coefficients as key factors influencing SOP reduction.
- We propose novel methods to achieve spatiotemporal spike sparsity: a multi-stage masking mechanism for spatial spike sparsity, which allows neurons to adopt variable firing thresholds, and temporal attention batch normalization (TABN), which prunes redundant spike activity across time steps to realize temporal spike sparsity.

[1]Our code: https://github.com/MmPple/SDJP

- We develop a dynamic joint pruning framework that comprehensively exploits the available redundancy for maximal SOP reduction. An dynamic strategy is introduced to adjust sparse regularization coefficients based on real-time computational impact.
- Experiments on CIFAR-10, CIFAR-100, and ImageNet demonstrate that our proposed framework significantly reduces SOPs and energy consumption while maintaining high model accuracy. On CIFAR-10, our method achieves an SOP compression ratio of up to 126.38× with only a 2.15% accuracy drop, significantly outperforming state-of-the-art methods.

II. Related Work and Background

A. Pruning for SNNs

Pruning induces sparsity in neural networks by removing certain elements, enhancing computational efficiency and reducing power consumption. In SNNs, the main forms of sparsity explored are weight sparsity and spike sparsity.

Weight Sparsity. Weight pruning in SNNs is widely studied because it enables low-power operation on neuromorphic hardware like Loihi [5], which supports unstructured sparsity. Inspired by pruning techniques in artificial neural networks (ANNs), magnitude-based pruning removes small weights during learning processes such as event-driven contrastive divergence [24], spike-timing-dependent plasticity [2], and ANN-to-SNN conversion [25]. Kim et al. [9] applied the lottery ticket hypothesis to SNN pruning using iterative magnitude pruning. ADMM combined spatio-temporal backpropagation with the alternating direction method of multipliers (ADMMs) [23]. Some incorporate biologically inspired strategies like synapse regeneration, such as Grad R [14] and ESLSNN [11].

Spike Sparsity. Sparse signals are inherently advantageous in SNNs, as sparse spiking patterns contribute to low-power computations in neuromorphic systems utilizing spike-triggered mechanisms. Pruning methods to achieve spike sparsity include regularization techniques and attention mechanisms. Regularization methods like L_p regularization [19] and low-activity regularization [20], [21], [23] promote high spike sparsity by eliminating redundancy without compromising performance. Attention mechanisms focus computational resources on critical information, reducing the number of spikes required for effective processing [18], [26], [27].

Hybrid Pruning. Some hybrid pruning methods have begun to combine sparsity at multiple levels. ADMM [23] performed weight pruning based on the ADMMs and activity regularization simultaneously. However, they simply combine the two without optimization, making it ineffective for joint pruning. UPSNN [22] combines unstructured weight and neuron pruning to maximize sparsity and enhance energy efficiency. However, UPSNN introduces neuron sparsity rather than finer-grained spike sparsity, and does not fully exploit redundancy to maximize energy efficiency.

B. Spiking Neural Network

Spiking neural networks apply the spiking neuron as the fundamental unit to collect signals from other neurons or inputs during a time step, and modulate the membrane potential by these signals. An output spike is generated and delivered to post-synaptic neurons when the membrane potential exceeds the threshold. The commonly used neuron model is Leaky Integrate and Fire (LIF) model [28]. It is defined as:

$$h[t] = u[t-1] + \frac{1}{\tau}(\sum_i w_i s_i[t] - u[t-1]), \tag{1}$$

$$u[t] = h[t] \cdot (1 - s[t]) + V_{reset} \cdot s[t], \tag{2}$$

where Eq.1 and Eq.2 describe the charging and resetting processes of a LIF neuron. t represents the current time step. s_i represents the spike train from the ith pre-synaptic neuron and w_i is the corresponding synaptic weight. $u[t]$ denotes the membrane potential at the end of the current time step t. V_{reset} represents the reset voltage, which we set to 0. $s[t] \in \{0, 1\}$ represents the output of the neuron in the current time step. The fire process is described as:

$$s[t] = \Theta(h[t] - V_{threshold}), \tag{3a}$$

$$\Theta(x) = 0, x < 0 \ otherwise \ 1, \tag{3b}$$

where $V_{threshold}$ is the threshold voltage.

For the learning process, we use the spatio-temporal backpropagation (STBP) [29] to train SNNs. To overcome the problem that Eq.3b is not differentiable at zero, we follow pervious works and use the arc tangent function as a surrogate function to replace Eq.3b in the backward phase [29].

III. Methodology

In this section, we present a framework for dynamic joint pruning in deep SNNs. Figure 1 shows an overview of the proposed framework. This framework performs sparse regularization with joint optimization to leverage spatiotemporal spike sparsity and weight sparsity. It integrates a sparse strategy that dynamically adjusts importance coefficients during sparse training. Our objective is to minimize SOPs in SNN inference while maintaining high model accuracy.

A. SOPs Model

The energy consumption in SNNs primarily arises from synaptic operations (SOPs). Specifically, the energy consumption for SNNs is given by:

$$E = C_E \cdot \#\text{SOPs} = C_E \cdot \sum_i s_i \cdot c_i, \tag{4}$$

where E represents the total energy consumption, C_E is the average energy required for each synaptic operation, s_i denotes the number of spikes fired by neuron i, and c_i is the number of synaptic connections from neuron i.

From Eq.4, reducing energy consumption depends on minimizing SOPs, which are determined by the number of spikes fired by each neuron (spike sparsity) and the number of synaptic connections (weight sparsity). To better describe spike sparsity, we incorporate the temporal dynamics of SNNs into the SOPs model:

$$\#\text{SOPs} = \sum_t \sum_i s_i[t] \cdot c_i, \tag{5}$$

where $s_i[t]$ denotes the number of spikes fired by neuron i at time step t. By considering time steps, we can further decompose spike sparsity across both time and space.

Based on Eq. 5, we define spatiotemporal spike sparsity, which describes the frequency and number of spikes fired by neurons across time and space. This provides a refined view of neuron activity, indicating the precise sparse pattern of spikes rather than simply whether a neuron is active. Furthermore, the SOPs model indicates that achieving high spatiotemporal spike sparsity and weight sparsity is critical for minimizing SOPs in SNN inference. This need to develop efficient joint pruning methods.

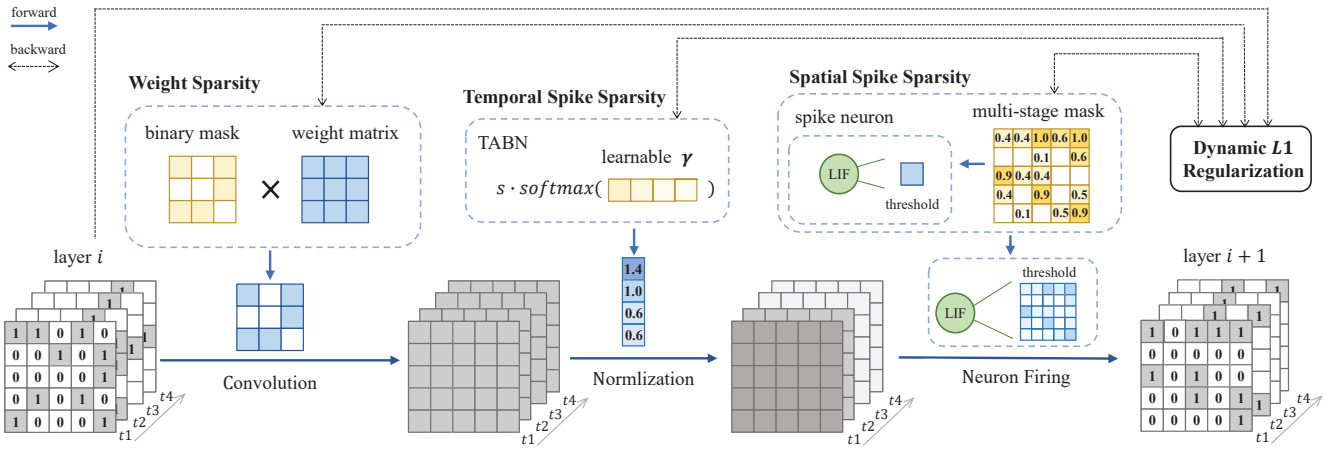

Fig. 1: The overview of proposed framework.

B. Spatial Spike Sparsity

To achieve high spike sparsity, L_1 regularization has proven effective [19]–[21]. However, this approach shows limited effectiveness when combined with weight pruning. Since L_1 regularization indirectly achieve spike sparsification through weight modification. This leads to a coupling between spike sparsity and weight sparsity.

To address this coupling issue, an intuitive solution is to independently reduce spikes through trainable gating parameters, such as neuron thresholds. However, the unbounded nature of threshold values, combined with the regularization constraint, poses significant challenges for training convergence. Thresholds may drift arbitrarily, destabilizing the optimization process. Moreover, completely pruning neurons would require manual threshold adjustments for each neuron, which is impractical and lacks scalability in large-scale networks.

We propose a novel multi-stage masking mechanism that equivalently substitutes variable thresholds in sparse training. Our approach applies a mask m_i^n to the neuron membrane potential $h_i[t]$:

$$h_i'[t] = m_i^n \cdot h_i[t] \tag{6}$$

where mask m_i^n is constrained to values from a set of K values uniformly distributed between 0 and 1. The multi-stage mask specifically addresses a key limitation of continuous masks, which typically converge toward a bimodal distribution of 0 and 1 during training, making it difficult to differentiate firing of neurons. This approach modulate membrane potentials through multiple stages, enabling fine-grained control over spike firing without complete neuron deactivation, thereby enhancing spatial spike sparsity.

To overcome the non-differentiability of step functions, we approximate the multi-stage mask by accumulating multiple sigmoid functions in practice:

$$m_i^n = \sum_{k=0}^{K-1} \frac{1}{K} \cdot \sigma\left(\beta_n \left(\theta_i - \frac{k}{K}\right)\right) \tag{7}$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ denotes the sigmoid function, β_n controls the function steepness, and θ_i represents the learnable parameter.

During the training process, θ_i are initialized to 0, and β_n is gradually increased, resulting in steeper sigmoid functions and increased difficulty for neurons to transition between stages. This gradual constraint allows neurons to naturally settle into different stages with corresponding firing thresholds.

During training, θ_i are initialized to 0 and trained with L_1 regularization, and β_n is gradually increased. This results in steeper sigmoid functions and makes it increasingly difficult for neurons to transition between stages. This gradual constraint allows neurons to naturally settle into different stages with corresponding firing thresholds.

C. Temporal Spike Sparsity

Recent studies have revealed significant redundancy in outputs across different time steps in SNN models—a phenomenon known as temporal information concentration [30] or spatio-temporal invariance [26]. This presents an opportunity for temporal spike sparsification. While existing temporal compression methods focus on directly reducing the number of time steps to decrease latency and computation, they are often ineffective for modern STBP-based SNNs that already operate with few time steps. Further reduction can lead to significant performance degradation.

To adaptively achieve temporal spike sparsity, we propose a novel data-independent temporal attention batch normalization (TABN) that incurs no additional computational cost during inference. TABN replaces the linear transformation in standard batch normalization layers with a temporal attention mechanism, enabling differentiated spike firing across time steps. The computation of TABN is formulated as:

$$v_i'[t] = s \cdot \left(\frac{v_i[t] - \mu}{\sqrt{\sigma^2 + \epsilon}}\right) \cdot \alpha_t + \beta, \tag{8}$$

where $v[t]$ represents the accumulated input of the i-th neuron at time step t, μ and σ denote the batch mean and standard deviation, and ϵ ensures numerical stability. The time step scaling factor α_t is normalized through a softmax function:

$$\alpha_t = \frac{e^{\gamma_t}}{\sum_{k=1}^{T} e^{\gamma_k}}, \tag{9}$$

where T represents the total number of time steps, and γ_t is a learnable parameter. The global learnable scaling factor s prevents spike and gradient vanishing during training, while β serves as a learnable bias term.

TABN learns temporal importance without introducing additional computational overhead. Specifically, we implement temporal spike sparsity at channel level, aligning with the observation that temporal output similarity manifests at the channel granularity [30]. Spike outputs of a channel represent a complete unit of feature expression and transmission. The channel-wise application of TABN enables

precise optimization of temporal redundancy while preserving feature integrity.

D. Dynamic Joint Pruning Framework

Combining proposed spatiotemporal spike sparsification mechanisms with L1 regularization, we develop a joint pruning framework comprising sparse training and fine-tuning phases.

In the sparse training phase, we jointly optimizes weight sparsity and spatiotemporal spike sparsity through the binary weight mask, multi-stage neuron mask, and TABN. We control the sparsity level using L1 regularization:

$$L_{\text{sparse}} = \lambda \left(\sum_i \|\boldsymbol{m}_i^w\|_1 + \sum_j \|m_j^n\|_1 + \sum_c \|\boldsymbol{\gamma}_c\|_1 \right) \tag{10}$$

where $\boldsymbol{m}_i^w$, m_j^n, and $\boldsymbol{\gamma}_c$ denote the weight mask vector for presynaptic neuron i, the multi-stage mask for neuron j, and the temporal weight vector for channel c, respectively. λ is a hyperparameter controlling the sparsity level. In practice, we use the approximation in Eq. 7 with $K = 1$ to overcome the non-differentiability of binary weight mask $\boldsymbol{m}_i^w$.

In the fine-tuning phase, we fix the sparse masks and convert multi-stage masks to neuron thresholds. The weights and neurons corresponding to zero masks with $\beta_n < 0$ are completely pruned. The learned temporal importance distribution of TABN is preserved. We retrain the pruned model without the sparsification loss L_{sparse} to recover performance loss induced by pruning during the sparse training phase.

To improve the joint pruning and minimize SOPs reduction, we introduce an strategy that dynamically adjusts sparse regularization terms based on real-time computational impact. According to the SOPs model Eq. 5, each sparsity form's contribution to computational efficiency is weighted by respective coefficients, leading to the dynamic sparse loss function:

$$L_{\text{sparse}} = \lambda \left(\sum_i s_i \|\boldsymbol{m}_i^w\|_1 + \sum_j w_j \|\boldsymbol{m}_j^n\|_1 + \sum_c w_c \|\boldsymbol{\gamma}_c\|_1 \right). \tag{11}$$

Here, the coefficients represent each sparsity form's computational impact:s_i measures spike count of presynaptic neuron i for weight sparsity, w_j reflects the preserved next-layer connection count of neuron j for spatial spike sparsity, and w_c indicates the remaining channel-level weights for temporal spike sparsity. These coefficients are dynamically updated during sparse training to ensure pruning decisions align with their actual computational efficiency impact.

For convolutional layers, we estimate coefficients at the channel level to simplify computation. Specifically, $\boldsymbol{m}_i^w$ represents weight masks associated with input channel i, encompassing all output channels and kernel spatial dimensions (height and width). The scaling factor s_i measures the spike count of input channel i. Similarly, the mask $\boldsymbol{m}_j^n$ corresponds to the multi-stage neuron mask vector for input channel j, and w_j reflects the number of preserved convolutional kernel weights associated with input channel j.

TABLE I: Hyperparameter settings for proposed method.

Dataset	Time Steps	Optimizer	Learning Rate	Batch Size
CIFAR10	8	Adam	0.0001	16
CIFAR100	4	SGD	0.025	64
ImageNet	4	Adam	0.001	256

IV. Experimental Results

A. Experimental Setup

In this section, we evaluate the proposed method for the image classification task on the CIFAR-10 [32], CIFAR-100 [32], and ImageNet [33] datasets. Following previous works, we employ spike frequency encoding and take the first convolutional layer of the model as a learnable encoder to transform static image inputs into spiking patterns. Our implementation is based on PyTorch and the SpikingJelly package. All experiments are executed on an RTX 3090 GPU.

We follow the hyperparameter setting from UPSNN [22]. For CIFAR-10, sparse training is performed over 800 epochs, followed by 200 fine-tuning epochs, while CIFAR-100 and ImageNet use 280 sparse training epochs and 40 fine-tuning epochs each. During the sparse training, β_n is linearly increased from 5 to 1000, with stage count K is fixed to 20. In the experiment of performance comparison, we use $\lambda = \{5, 10, 40\} \times 10^{-12}$ for CIFAR-10, $\{1, 2, 3\} \times 10^{-9}$ for CIFAR-100, and $\{1, 6, 10\} \times 10^{-11}$ for ImageNet. The details of other hyperparameters is shown in Table I.

B. Comparison with the State-of-the-Art Works

1) Performance Comparison: We compare our method with state-of-the-art SNN pruning algorithms including ADMM [23], Grad R [14], ESLSNN [11], STDS [31], and UPSNN [22], as summarized in Table II. In the table, "Avg. SOPs" indicates average SOPs per instance in inference. "Ratio" is the compression ratio, indicating the energy efficiency. "Sparsity Type" specifies the achieved sparsity: "W" for weight sparsity, "W+N" for weight and neuron sparsity, and "W+S" for weight and spike sparsity.

On CIFAR-10, our method achieves the highest accuracy of 92.65% while reducing energy consumption to 21.55M SOPs and achieving a compression ratio of 36.24. This outperforms UPSNN, which requires 38.32M SOPs with a ratio of 20.25 for similar accuracy. At higher sparsity levels, our method maintains superior accuracy and further reduces energy consumption, demonstrating up to 126.38× improvement in energy efficiency. For CIFAR-100, our method attains an accuracy of 72.51%, using only 17.11M SOPs, surpassing both STDS and UPSNN in accuracy and energy savings. On the more challenging ImageNet dataset, our method achieves 61.96% accuracy, using only 227.45M SOPs, outperforming previous methods that either have lower accuracy or require more SOPs. These results demonstrate that by jointly pruning weights and spikes, our method effectively balances high accuracy with significant energy reduction across various datasets.

2) Sparsity Comparison: Table III presents the effect of joint pruning in achieving spike sparsity and weight sparsity at different accuracy levels. We compare proposed method with UPSNN [22] and ADMM [23] on SEW ResNet18 for CIFAR-100. In Table III, "#Spike" and "Avg. FR" denote the average number of spikes and the average spike firing rate per neuron across instances in test set. "Param." indicates the total number of remaining weights after pruning. Our method achieves the best performance among the compared methods. Compared to UPSNN, our method significantly reduces both the total number of spikes and the average firing rate. This reduction is attributed to our fine-grained control over neuron spike firing. These results demonstrate the efficiency of our method in joint pruning for spike and weight sparsity.

C. Ablation Study Results

1) Performance with different SOPs: We evaluate the robustness of proposed method across various average SOPs. As depicted in Fig-

TABLE II: Performance comparison between proposed method and the state-of-the-art works. We mark the best results in bold.

Dataset	Method	Arch.	Sparsity Type	Test Acc. (%)	Acc. Loss (%)	Avg. SOPs (M)	Ratio
CIFAR10	Grad R [14]	6Conv, 2FC	W	92.54	-0.30	371.05	2.09
				92.03	-0.81	143.69	5.40
				89.32	-3.52	41.89	18.53
	ESLSNN [11]	ResNet19	W	92.08	-0.63	298.24	2.11
				91.46	-1.25	178.10	3.54
				90.90	-1.81	108.89	5.79
	ADMM [23]	7Conv, 2FC	W+S	90.19	-0.13	107.97	2.91
				88.18	-2.14	49.72	6.32
				79.63	-10.69	28.65	10.96
	UPSNN [22]	6Conv, 2FC	W+N	92.63	-0.21	38.32	20.25
				92.05	-0.79	16.47	47.12
				90.65	-2.19	8.50	91.31
	This Work	6Conv, 2FC	W+S	**92.65**	**-0.21**	**21.55**	**36.24**
				92.18	**-0.68**	**9.61**	**81.27**
				90.71	**-2.15**	**6.18**	**126.38**
CIFAR100	STDS [31]	SEW ResNet18	W	72.69	-1.36	48.65	5.34
				71.05	-3.00	19.11	13.61
				70.13	-3.92	13.90	18.71
	UPSNN [22]	SEW ResNet18	W+N	72.34	-1.82	27.16	9.24
				71.43	-2.73	21.16	11.86
				70.45	-3.71	9.60	26.14
	This Work	SEW ResNet18	W+S	**72.51**	**-1.54**	**17.11**	**15.20**
				71.77	**-2.28**	**9.96**	**26.10**
				70.69	**-3.36**	**6.81**	**38.18**
ImageNet	STDS [31]	SEW ResNet18	W	61.51	-1.67	512.95	2.61
				59.93	-3.25	273.39	4.89
				58.06	-5.12	182.49	7.33
	UPSNN [22]	SEW ResNet18	W+N	61.89	-1.29	311.70	4.29
				60.00	-3.18	177.99	7.52
				58.99	-4.19	116.88	11.45
	This Work	SEW ResNet18	W+S	**61.96**	**-1.14**	**227.45**	**5.88**
				60.31	**-2.79**	**109.18**	**12.24**
				59.07	**-4.03**	**57.31**	**23.33**

TABLE III: Sparsity comparison between proposed method and the state-of-the-art works. We mark best results in bold.

Method	Test Acc.(%)	#Spike	Avg. FR	Param. (M)
ADMM [23]	72.50	250647	0.1141	5.75
	71.65	215339	0.0986	3.89
UPSNN [22]	72.47	261386	0.1630	3.93
	71.59	165418	0.1419	3.11
This Work	**72.51**	**79321**	**0.0470**	**3.19**
	71.77	**56491**	**0.0439**	**2.72**

ure 2a, our method consistently maintains high test accuracy on the CIFAR-100 dataset even as the average SOPs decrease significantly. When reducing the average SOPs from 38.23M to 3.82M, which is a reduction of over 10 times, the test accuracy only declines from 73.72% to 69.12%. These results highlight that our method remains effective across a wide range of SOP levels, especially at lower SOPs corresponding to lower energy costs.

2) Effect of Different Mechanisms: To understand the impact of each mechanism in our method, we perform an ablation study by individually removing the multi-stage mask, temporal attention batch normalization (TABN), and the dynamic coefficient strategy. As shown in Figure 2a, the absence of the multi-stage mask results in a sharper accuracy loss as SOPs decrease, indicating that spatial spike sparsity plays a crucial role in reducing SOPs. Removing TABN and the dynamic coefficient strategy also affects accuracy, especially at higher SOPs levels. This is because, at this point, there is more redundant spike activity across time steps and greater variance in the importance among different sparsity terms, offering more opportunity for effective sparsity.

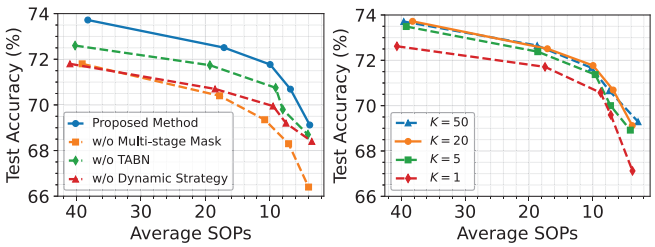

Fig. 2: (a) Results of pruned models with different mechanisms for CIFAR-100. (b) Results of pruned models fro CIFAR-100 with different stage count K in the multi-stage mask.

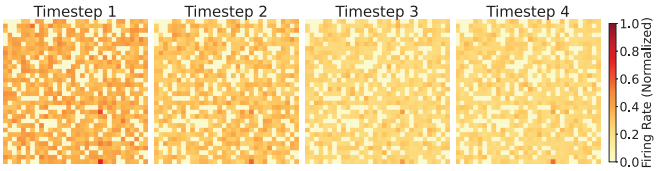

Fig. 3: Visualization of spatiotemporal spike sparsity in feature maps of the penultimate layer of pruned model on CIFAR-100.

3) Effect of Stage Count K: We evaluate the impact of different stage count settings K in the multi-stage mask on the performance of our method. As shown in Figure 2b, when $K = 1$, the multi-stage mask reduces to a binary mask, leading to suboptimal performance. As K increases, the performance improves, demonstrating the benefit of using a multi-stage mask with finer granularity. In particular, our method exhibits stable performance across a wide range of K values (from 5 to 50), indicating that it is robust to the choice of stage count.

4) Visualization of Spatiotemporal Spike Sparsity: To further illustrate the effect of proposed method, we visualize the spatiotemporal spike sparsity in the feature map outputs of the penultimate layer of pruned model on CIFAR-100, As shown in Figure 3, the color intensity indicates the average neuron firing rate. The visualization reveals both spatial and temporal spike sparsity achieved by our method. Spatially, at a single time step, neurons exhibit significant variations in firing rates, demonstrating that the multi-stage mask effectively promotes sparsity by selectively suppressing less important neurons. Temporally, we observe that spikes are mainly concentrated in the first two time steps. This is attributed to the TABN, which reduces redundant spike activity over time.

V. Conclusion

In this paper, we propose a dynamic joint pruning framework for SNNs that fully exploits weight sparsity and spatiotemporal spike sparsity to maximizing SOPs reduction and energy efficiency in inference. Our multi-stage masking mechanism and temporal attention batch normalization enable fine-grained control over spatiotemporal neuron activity without compromising model accuracy. Experimental results demonstrate that our framework significantly reduces SOPs and energy consumption, achieving superior performance compared to state-of-the-art methods. This work highlights the importance of jointly optimizing multiple forms of sparsity and provides a foundation for future research in low-power SNN computing.

Acknowledgments

This work was supported by the Postdoctoral Fellowship Program (Grade B) of China Postdoctoral Science Foundation under grant number GZB20240760.

REFERENCES

[1] W. Maass, "Networks of spiking neurons: the third generation of neural network models," *Neural networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[2] N. Rathi, P. Panda, and K. Roy, "Stdp-based pruning of connections and weight quantization in spiking neural networks for energy-efficient recognition," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 4, pp. 668–677, 2018.

[3] A. Basu, L. Deng, C. Frenkel, and X. Zhang, "Spiking neural network integrated circuits: A review of trends and future directions," in *2022 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 2022, pp. 1–8.

[4] F. Akopyan, J. Sawada, A. Cassidy, R. Alvarez-Icaza, J. Arthur, P. Merolla, N. Imam, Y. Nakamura, P. Datta, G.-J. Nam, B. Taba, M. Beakes, B. Brezzo, J. B. Kuang, R. Manohar, W. P. Risk, B. Jackson, and D. S. Modha, "Truenorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip," *IEEE transactions on computer-aided design of integrated circuits and systems*, vol. 34, no. 10, pp. 1537–1557, 2015.

[5] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C.-K. Lin, A. Lines, R. Liu, D. Mathaikutty, S. McCoy, A. Paul, J. Tse, G. Venkataramanan, Y.-H. Weng, A. Wild, Y. Yang, and H. Wang, "Loihi: A neuromorphic manycore processor with on-chip learning," *Ieee Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[6] J. Pei, L. Deng, S. Song, M. Zhao, Y. Zhang, S. Wu, G. Wang, Z. Zou, Z. Wu, W. He, F. Chen, N. Deng, S. Wu, Y. Wang, Y. Wu, Z. Yang, C. Ma, G. Li, W. Han, H. Li, H. Wu, R. Zhao, Y. Xie, and L. Shi, "Towards artificial general intelligence with hybrid tianjic chip architecture," *Nature*, vol. 572, no. 7767, pp. 106–111, 2019.

[7] H. Zheng, Y. Wu, L. Deng, Y. Hu, and G. Li, "Going deeper with directly-trained larger spiking neural networks," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 12, 2021, pp. 11 062–11 070.

[8] W. Fang, Z. Yu, Y. Chen, T. Huang, T. Masquelier, and Y. Tian, "Deep residual learning in spiking neural networks," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 21 056–21 069.

[9] Y. Kim, Y. Li, H. Park, Y. Venkatesha, R. Yin, and P. Panda, "Exploring lottery ticket hypothesis in spiking neural networks," in *Computer Vision–ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XII*. Springer, 2022, pp. 102–120.

[10] S. Huang, H. Fang, K. Mahmood, B. Lei, N. Xu, B. Lei, Y. Sun, D. Xu, W. Wen, and C. Ding, "Neurogenesis dynamics-inspired spiking neural network training acceleration," in *60th ACM/IEEE Design Automation Conference, DAC*, 2023, pp. 1–6.

[11] J. Shen, Q. Xu, J. Liu, Y. Wang, G. Pan, and H. Tang, "Esl-snns: An evolutionary structure learning strategy for spiking neural networks," in *AAAI Conference on Artificial Intelligence*, 2023.

[12] J. Chen, H. Yuan, J. Tan, B. Chen, C. Song, and D. Zhang, "Resource constrained model compression via minimax optimization for spiking neural networks," in *Proceedings of the 31st ACM International Conference on Multimedia*, 2023, p. 5204–5213.

[13] G. Bellec, D. Salaj, A. Subramoney, R. A. Legenstein, and W. Maass, "Long short-term memory and learning-to-learn in networks of spiking neurons," *Advances in neural information processing systems*, vol. 31, 2018.

[14] Y. Chen, Z. Yu, W. Fang, T. Huang, and Y. Tian, "Pruning of deep spiking neural networks through gradient rewiring," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, Z.-H. Zhou, Ed. International Joint Conferences on Artificial Intelligence Organization, 8 2021, pp. 1713–1721, main Track.

[15] D. Wu, Y. Qi, K. Cai, G. Jin, X. Yi, and X. Huang, "Direct training needs regularisation: Anytime optimal inference spiking neural network," 2024. [Online]. Available: https://arxiv.org/abs/2405.00699

[16] Q. Meng, M. Xiao, S. Yan, Y. Wang, Z. Lin, and Z.-Q. Luo, "Training high-performance low-latency spiking neural networks by differentiation on spike representation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 12 444–12 453.

[17] N. Rathi and K. Roy, "Diet-snn: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 6, pp. 3174–3182, 2023.

[18] M. Yao, G. Zhao, H. Zhang, Y. Hu, L. Deng, Y. Tian, B. Xu, and G. Li, "Attention spiking neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 8, pp. 9393–9410, 2023.

[19] S. Narduzzi, S. A. Bigdeli, S.-C. Liu, and L. A. Dunbar, "Optimizing the consumption of spiking neural networks with activity regularization," in *2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 61–65.

[20] D. Neil, M. Pfeiffer, and S.-C. Liu, "Learning to be efficient: algorithms for training low-latency, low-compute deep spiking neural networks." New York, NY, USA: Association for Computing Machinery, 2016. [Online]. Available: https://doi.org/10.1145/2851613.2851724

[21] T. Pellegrini, R. Zimmer, and T. Masquelier, "Low-activity supervised convolutional spiking neural networks applied to speech commands recognition," in *2021 IEEE Spoken Language Technology Workshop (SLT)*, 2021, pp. 97–103.

[22] X. Shi, J. Ding, Z. Hao, and Z. Yu, "Towards energy efficient spiking neural networks: An unstructured pruning framework," in *The Twelfth International Conference on Learning Representations*, 2024.

[23] L. Deng, Y. Wu, Y. Hu, L. Liang, G. Li, X. Hu, Y. Ding, P. Li, and Y. Xie, "Comprehensive snn compression using admm optimization and activity regularization," *IEEE transactions on neural networks and learning systems*, 2021.

[24] E. O. Neftci, B. U. Pedroni, S. Joshi, M. Al-Shedivat, and G. Cauwenberghs, "Stochastic synapses enable efficient brain-inspired learning machines," *Frontiers in neuroscience*, vol. 10, p. 241, 2016.

[25] Y. Liu, K. Qian, S. Hu, K. An, S. Xu, X. Zhan, J. J. Wang, R. Guo, Y. Wu, T.-P. Chen, Q. Yu, and Y. Liu, "Application of deep compression technique in spiking neural network chip," *IEEE transactions on biomedical circuits and systems*, vol. 14, no. 2, pp. 274–282, 2019.

[26] M. Yao, J. Hu, G. Zhao, Y. Wang, Z. Zhang, B. Xu, and G. Li, "Inherent redundancy in spiking neural networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 16 924–16 934.

[27] S. Kundu, G. Datta, M. Pedram, and P. A. Beerel, "Spike-thrift: Towards energy-efficient deep spiking neural networks by limiting spiking activity via attention-guided compression," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, January 2021, pp. 3953–3962.

[28] E. M. Izhikevich, "Simple model of spiking neurons," *IEEE Transactions on neural networks*, vol. 14, no. 6, pp. 1569–1572, 2003.

[29] Y. Wu and et.al., "Spatio-temporal backpropagation for training high-performance spiking neural networks," *Frontiers in neuroscience*, vol. 12, p. 331, 2018.

[30] Y. Kim, Y. Li, H. Park, Y. Venkatesha, A. Hambitzer, and P. Panda, "Exploring temporal information dynamics in spiking neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 7, 2023, pp. 8308–8316.

[31] Y. Chen, Z. Yu, W. Fang, Z. Ma, T. Huang, and Y. Tian, "State transition of dendritic spines improves learning of sparse spiking neural networks," in *International Conference on Machine Learning*, 2022.

[32] A. Krizhevsky, "Learning multiple layers of features from tiny images," 2009.

[33] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.